

ИПС «Арахна»: что умеет система

Лабораторная работа № 1 · вариант 3 · информационно-поисковая система с элементами искусственного интеллекта в интерфейсе

«Арахна» ищет документы в локальной вычислительной сети по запросу на обычном русском языке. Паук обходит папки компьютеров и сетевые ресурсы вида \\сервер\папка, извлекает текст из файлов TXT, MD, CSV, HTML, DOCX, PDF и RTF, приводит слова к начальной форме и строит индекс. По запросу система возвращает документы, упорядоченные по близости к нему: это векторная модель с весами TF-IDF и косинусной мерой. Суть варианта 3 — элементы искусственного интеллекта в интерфейсе: система показывает, как поняла запрос, исправляет опечатки, добавляет синонимы, учится на отметках «больше таких» и «не то» и объясняет, почему документ найден. Отдельные разделы оценивают качество поиска метриками РОМИП.

Зачем это нужно: среди тысяч файлов на разных компьютерах отдела нужный документ находится по смыслу, даже если не помнишь ни имени файла, ни папки, — и видно, насколько хорошо система с этим справляется.

В системе семь рабочих страниц. Кроме них есть «Справка» и шесть игровых разделов паука Пафнутия («Достижения», «Лавка», «Слепая выдача», «Блиц», «Пасьянс», «Заявление») — на поиск и метрики они не влияют и ниже не описываются. Запуск: `python run.py`, адрес <http://127.0.0.1:8000/>.

Страница	Адрес	Для чего
Поиск	/	ввести запрос, задать параметры
Результаты поиска	/search?q=...	выдача по рангу, разбор запроса, объяснение, отметки
Документ	/doc/<номер>	текст, ключевые слова, похожие документы, исходный файл
Индекс	/admin	ресурсы ЛВС, обход и индексация, статистика
Оценка качества	/metrics	метрики РОМИП, графики, выгрузка
Сравнение конфигураций	/metrics/compare	что дают морфология, синонимы, строгий режим
Разметка	/qrels	эталонные оценки релевантности

1. Поиск <http://127.0.0.1:8000/>

На первой странице — строка запроса и «Параметры поиска»: «все слова запроса обязательны», «расширять запрос синонимами», «учитывать мои отметки релевантности» и фильтр по дате файла. Ниже — состояние системы (документы, термины в словаре, веса в индексе, узлы ЛВС, число заданных запросов) и списки частых и последних запросов, которые можно повторить одним щелчком. Пока пользователь набирает запрос, под строкой появляются подсказки.

Пример. Набираем «марш» — система подсказывает «маршрутизация пакетов между узлами сети» и «маршрутизация пакетов» (прошлые запросы), затем «маршрут», «маршрутизатор», «маршрутизаторе» (словоформы из индекса).

Состояние: 108 документов, 2 437 терминов, 5 718 весов, 3 узла ЛВС.

Как это работает. Автодополнение берёт варианты из двух источников — журнала прошлых запросов и словаря словоформ индекса — по совпадению начала и упорядочивает их по частоте. Подсказки подгружаются без перезагрузки страницы.

2. Результаты поиска /search?q=...

Выдача показывает, сколько документов найдено и за какое время, и список документов по убыванию ранга. У каждого результата есть заголовок (ссылка на карточку документа), ранг — значение косинусной меры, узел ЛВС, дата и путь к файлу. Путь — активная ссылка на исходный файл. Ниже — фрагмент текста с подсвеченными словами и строка «Слова запроса в документе». Под результатом — «Почему найден» (вклад каждого термина в ранг), кнопки «Больше таких», «Не то» и «Похожие».

Над списком — блок «Как система поняла запрос»: нормальная форма, часть речи и наличие в словаре для каждого слова, отброшенные стоп-слова и добавленные синонимы. При опечатке система пишет «Возможно, вы имели в виду...», при пустой выдаче — что изменить в запросе. Необязательный языковой помощник (любой сервис с OpenAI-совместимым API) может кратко ответить по найденным документам. По умолчанию он выключен и в ранжировании не участвует.

Пример. Запрос «маршрутизация пакетов между узлами сети». «Между» — стоп-слово; леммы: маршрутизация, пакет, узел, сеть; синонимы: узел → сервер, хост; сеть → лвс, сетевой. Найдено 52 документа за ≈ 94 мс:

1) «Маршрутизаторы и маршрутизация пакетов» (NODE-A) — $0,3119 = 0,1359$
(маршрутизация) + 0,0794 (пакет) + 0,0688 (сеть) + 0,0277 (узел);

2) «IP-адресация и разбиение на подсети» (NODE-C) — 0,1875;

3) «VPN и удалённый доступ к ресурсам предприятия» (NODE-A) — 0,1648.

Запрос «коммутатор сети» → «Возможно, вы имели в виду: коммутатор сети».

Как это работает. Слова запроса приводятся к лемме морфологическим анализатором `rumorphy3` («узлами» → «узел»), стоп-слова отбрасываются. Из лемм строится поисковый образ запроса (ПОЗ) — вектор, где $w_{qj} = 1$ для слов пользователя и 0,5 для добавленных системой (синонимов и исправлений опечаток). Ранг документа — косинус угла между его вектором и вектором запроса:

$$\text{sim}(D, Q) = \sum_j A_{ij} \cdot w_{qj} / (\|D\| \cdot \|Q\|), \quad \|D\| = \sqrt{\sum_j A_{ij}^2}, \quad \|Q\| = \sqrt{\sum_j w_{qj}^2}$$

где A_{ij} — вес термина j в документе i (см. страницу «Документ»). Деление на нормы убирает влияние длины документа, и остаётся «направление» вектора, то есть тема. В примере $\|Q\| = \sqrt{(4 \cdot 1^2 + 4 \cdot 0,5^2)} = \sqrt{5}$. Вклад термина в ранг — $A_{ij} \cdot w_{qj} / (\|D\| \cdot \|Q\|)$; сумма вкладов равна рангу. Режим «все слова обязательны» превращает поиск в логическое И.

- **Опечатки** — расстояние Дамерау — Левенштейна: наименьшее число вставок, удалений, замен и перестановок соседних букв, превращающих одно слово в другое. «комутатор» → «коммутатор» — 1 (вставка); «оепрационная» → «операционная» — 1 (перестановка, у Левенштейна было бы 2). Порог — 1 для слов до 5 букв и 2 для более длинных; при равном расстоянии выбирается более частое слово.
- **Синонимы** — тезаурус предметной области из 36 групп (коммутатор — свитч, маршрутизатор — роутер, резервный — бэкап...).

- «Больше таких» / «Не то» — обратная связь по релевантности, метод Рокчио: вектор запроса сдвигается к отмеченным удачным документам и от неудачных:

$$Q' = \alpha \cdot Q + \beta / |D_r| \cdot \sum_{d \in D_r} d / \|d\| - \gamma / |D_{nr}| \cdot \sum_{d \in D_{nr}} d / \|d\|$$

где D_r — документы с отметкой «больше таких», D_{nr} — «не то»; $\alpha = 1$, $\beta = 0,75$, $\gamma = 0,15$. Отрицательные веса обнуляются, в векторе остаются 40 самых весомых терминов.

3. Документ /doc/<номер>

Карточка документа: заголовок, узел ЛВС, дата добавления, формат, размер, число значимых слов; кнопка «Открыть исходный файл» (активная ссылка /raw/<номер>) и путь в сети; текст с подсвеченными словами запроса. Ниже — «Поисковый образ документа» (автоматически выделенные ключевые слова с весами) и «Похожие документы». Документ можно удалить из базы.

Пример. «Маршрутизаторы и маршрутизация пакетов»: маршрутизатор 5,725; маршрутизация 4,067; подсеть 3,113; адрес 2,863; пакет 2,377; сеть 2,060. Похожие: «Автоматическая настройка адресов по DHCP» — 0,162; «IP-адресация и разбиение на подсети» — 0,158; «Межсетевой экран и фильтрация трафика» — 0,144.

Как это работает. Ключевые слова выделяются автоматически по формулам методички:

$$B_i = \log (N / P_i) \quad (1.5) \quad A_{ij} = Q_{ij} \cdot B_i \quad (1.6)$$

где N — число документов, P_i — число документов с термином i , B_i — инверсная частота термина, Q_{ij} — частота термина i в документе j , A_{ij} — его вес; логарифм десятичный. Термин, который есть во всех документах, получает $B = 0$ и ни на что не влияет, а редкий — наибольший вес. В примере $N = 108$: «маршрутизация» встречается в одном документе, $B = \lg 108 = 2,033$, и дважды в нём, $A = 2 \cdot 2,033 = 4,067$; «маршрутизатор» — в четырёх документах, $B = \lg 27 = 1,431$, и четырежды в этом, $A = 5,725$. «Похожие документы» — косинус между векторами документов по 25 самым весомым терминам образца; выводятся 5 ближайших.

4. Индекс /admin

Здесь управляют тем, что обходит паук. «Ресурсы под наблюдением» — каталоги и сетевые папки (\\сервер\папка) с датой последнего обхода; ресурс можно добавить или убрать, а кнопка «Добавить тестовые узлы ЛВС» подключает демонстрационную коллекцию. Есть кнопки «Запустить обход и индексацию», «Перестроить индекс» и «Очистить базу»; ход работы виден по полосе прогресса. Ниже — статистика базы, документы по узлам и форматам, частотный словарь (термин, P_i , ΣQ_{ij}) и журнал обходов (добавлено, обновлено, пропущено, удалено, ошибок).

Пример. 108 документов на трёх узлах (NODE-A, NODE-B, NODE-C — по 36); форматы: TXT 22, HTML 22, DOCX 22, MD 21, PDF 21; 2 437 терминов, 3 946 словоформ, 5 718 весов; в среднем 64,1 значимых слова на документ; объём 1 093 КиБ. Самые частые термины: «сеть» (в 33 документах, 81 раз), «система» (33, 63), «данные» (30, 47). Повторный обход неизменившейся коллекции ничего не пересчитывает.

Как это работает. Паук (агент) рекурсивно обходит каталоги, пропуская служебные папки и неподдерживаемые форматы. Обход инкрементальный: если время изменения и размер файла совпадают с сохранёнными, файл не читается; у изменившегося файла считается хеш SHA-1, и если он прежний, обновляется только дата. Изменившиеся файлы читаются восемью потоками. Текст извлекается по формату, слова лемматизируются, стоп-слова отбрасываются. Индекс перестраивается целиком: новый документ меняет N , а значит все B_i и A_{ij} . Хранится индекс в SQLite: documents — документы и нормы $\|D\|$, terms — словарь и P_i , postings — Q_{ij} и A_{ij} (инвертированный индекс). Поиск выполняется одним SQL-запросом, тексты документов при этом не читаются.

5. Оценка качества /metrics

Выдача по набору эталонных запросов сравнивается с оценками эксперта. Вверху задаются глубина выдачи top-K и флажки (синонимы, строгий режим); кнопки — «Пересчитать», «Сравнить конфигурации», «Выгрузить CSV», «Сохранить графики в PNG», «Загрузить эталон». Ниже — восемь карточек с метриками, 11-точечный график «точность — полнота», столбцы средней точности каждого запроса с линией MAP и таблица метрик по запросам.

Пример. 36 информационных потребностей, 271 оценка, top-20: MAP 0,634; P@5 0,467; P@10 0,286; R-точность 0,568; полнота 0,832; F1 0,371; nDCG 0,859; bpref 0,649.

Запрос «настройка локальной вычислительной сети»: релевантных 9, в top-20 найдено 8 → $P = 8/20 = 0,400$; $R = 8/9 = 0,889$; $P@5 = 0,8$; R-точность 0,667; AP 0,687; nDCG 0,900; bpref 0,593.

На графике точность не ниже 0,78 до полноты 0,5 и 0,26 — при полной полноте. Невысокая общая точность (0,27) — не недостаток: у запроса в среднем 4,1 релевантных документа, и даже идеальная система на глубине 20 дала бы около 0,2.

Как это работает. Метрики приняты в РОМИП (Российском семинаре по оценке методов информационного поиска). rel — найденные документы, rel — релевантные:

- **точность и полнота:** $P = |\text{rel} \cap \text{ret}| / |\text{ret}|$, $R = |\text{rel} \cap \text{ret}| / |\text{rel}|$; **F-мера** — их гармоническое среднее $F = 2PR / (P + R)$;
- **P@n** — точность первых n документов; **R-точность** — $P@R$, где R — число релевантных;
- **средняя точность AP** $= (1 / |\text{rel}|) \cdot \sum_k P@k \cdot \text{rel}(k)$, где $\text{rel}(k) = 1$, если на позиции k релевантный документ; **MAP** — среднее AP по запросам, главная метрика ранжирования;
- **11-точечный график** — интерполированная точность $P_{\text{int}}(r) = \max_{r' \geq r} P(r')$ на уровнях полноты 0; 0,1; ...; 1, усреднённая по запросам;
- **nDCG** $= \text{DCG} / \text{IDCG}$, $\text{DCG} = \sum_i (2^{\text{rel}(i)} - 1) / \log_2(i + 1)$: учитывает степень релевантности (оценка 2 важнее 1) и то, насколько высоко документ стоит; IDCG — DCG идеального порядка;
- **bpref** $= (1 / R) \cdot \sum_r (1 - \min(n_r, R) / \min(R, N))$: n_r — оценённые нерелевантные документы выше релевантного r, N — всего оценённых нерелевантных. Учитывает только оценённые документы, поэтому устойчива к неполной разметке.

6. Сравнение конфигураций /metrics/compare

Один и тот же набор запросов прогоняется при разных настройках: результаты сведены в таблицу, а кривые «точность — полнота» всех конфигураций — на один график. Страница показывает, что именно даёт каждая часть системы.

Пример. Базовая (леммы, бинарный ПОЗ): MAP 0,634, nDCG 0,859, bpref 0,649.

С расширением синонимами: 0,670 / 0,876 / 0,697.

Без морфологии (словоформы): 0,564 / 0,808 / 0,601.

Строгий режим (все слова обязательны): 0,242 / 0,475 / 0,247.

Как это работает. Для каждой конфигурации заново строится выдача и считаются те же метрики, что на странице «Оценка качества». Морфология повышает MAP на 12 % (0,564 → 0,634): без неё «сети» и «сетей» — разные термины. Синонимы добавляют ещё 0,036. Строгий режим превращает векторную модель в логическую (документ обязан содержать все слова) и снижает MAP в 2,6 раза: для запросов на естественном языке векторная стратегия лучше.

7. Разметка /qrels

Здесь создаётся эталон, по которому считаются метрики. Слева — информационные потребности (сколько документов оценено и сколько из них релевантных), поле для новой потребности, загрузка и сохранение CSV. Если выбрать запрос, система покажет первые документы своей выдачи, и каждому ставится оценка: 2 — точно отвечает, 1 — частично, 0 — не по теме. Метрики этого запроса пересчитываются сразу: видно, что изменилось и какой вклад запрос вносит в общий MAP. Запрос, у которого восемь и более оценок и ни одной нулевой, помечается как подозрительный.

Пример. В эталоне 36 потребностей и 271 оценка. Для «настройка локальной вычислительной сети» размечено 9 релевантных документов; поставив оценку, сразу видишь, как изменились P, R, AP, nDCG и bpref.

Как это работает. Оценки хранятся в базе и в data/qrels.csv по имени файла, поэтому повторный обход сети разметку не ломает. Нулевая оценка так же важна, как положительная: без явно нерелевантных документов bpref вырождается в единицу (сравнивать не с чем), а точность завышается. Метрики всегда считаются по «чистой» конфигурации — без отметок пользователя и игровых надстроек.